JavaScript Promises

A **Promise** is one of the most important concepts in JavaScript.

It is used for:

- API calls
- database queries
- authentication
- payment gateways
- file reading
- async operations

Modern JavaScript is heavily based on promises.

First Understand the Problem

Before promises, JavaScript used:

Callbacks

Example:

```
setTimeout(() => {
  console.log("Data received");
}, 2000);
```

Callbacks are functions passed into another function.

Callback Hell Problem

Imagine:

```
loginUser(() => {

  getProfile(() => {

    getPosts(() => {

      getComments(() => {

        console.log("Done");

      });

    });

  });

});
```

This becomes:

- unreadable
- difficult to debug
- difficult error handling
- deeply nested

This is called:

Callback Hell / Pyramid of Doom

Promises were introduced to solve this.

What is a Promise?

A promise is an object representing:

```
A future value
```

The value may be:

- available now
- available later
- never available

Real Life Example

Suppose you ordered food online.

Restaurant says:

```
"We promise to deliver your food."
```

Now 3 things can happen:

State	Meaning
Pending	food preparing
Fulfilled	delivered
Rejected	cancelled

This is exactly how promises work.

Promise States

A promise has ONLY 3 states.

---

1. Pending

Initial state.

```
const promise = new Promise(() => {});
```

Still waiting.

---

2. Fulfilled (Resolved)

Operation successful.

```
resolve("Success");
```

---

3. Rejected

Operation failed.

```
reject("Error");
```

Very Important Rule

A promise can change state only ONCE.

```
Pending → Fulfilled
OR
Pending → Rejected
```

Never both.

Promise Syntax

```
const promise = new Promise((resolve, reject) => {

});
```

The function inside is called:

Executor Function

It runs immediately.

Two Special Functions

Function	Purpose
resolve()	success
reject()	failure

First Promise Example

```
const myPromise = new Promise((resolve, reject) => {

  let success = true;

  if (success) {
    resolve("Login successful");
  } else {
    reject("Login failed");
  }

});

console.log(myPromise);
```

Output

```
Promise { 'Login successful' }
```

OR

```
Promise { <rejected> 'Login failed' }
```

Promise Consumers

To use promise result:

Method	Purpose
then()	success
catch()	error
finally()	always

---

then()

Runs after resolve.

```
promise.then((data) => {
  console.log(data);
});
```

---

catch()

Runs after reject.

```
promise.catch((error) => {
  console.log(error);
});
```

---

finally()

Runs always.

```
promise.finally(() => {
  console.log("Finished");
});
```

Full Example

```
const promise = new Promise((resolve, reject) => {

  setTimeout(() => {

    let success = true;

    if (success) {
      resolve("Data fetched");
    } else {
      reject("Server error");
    }

  }, 2000);

});

promise
  .then((data) => {
    console.log("Success:", data);
  })
  .catch((error) => {
    console.log("Error:", error);
  })
  .finally(() => {
    console.log("Completed");
  });
```

Output After 2 Seconds

```
Success: Data fetched
Completed
```

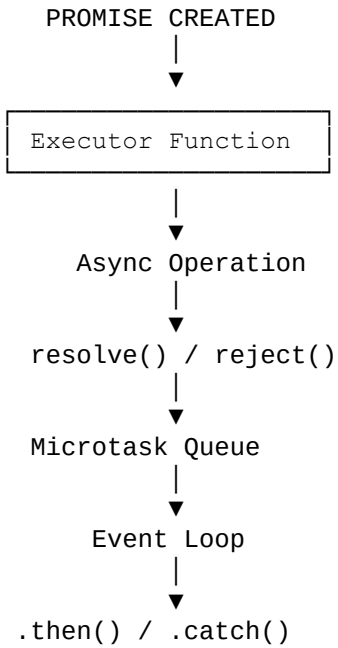
Internal Working of Promise

This is VERY IMPORTANT.

Promise Execution Flow

```
1. Promise created
2. Executor function runs immediately
3. Async task goes to Web APIs
4. Promise stays pending
5. Task completes
6. resolve/reject called
7. Callback enters Microtask Queue
8. Event loop executes callback
```

Visual Diagram



Most Important Point

Promise callbacks go into:

Microtask Queue

NOT callback queue.

That is why promises execute before setTimeout.

Example

```
console.log("1");

setTimeout(() => {
  console.log("2");
}, 0);

Promise.resolve().then(() => {
  console.log("3");
});

console.log("4");
```

Output

```
1
4
3
2
```

Why?

Because:

Queue	Priority
Microtask Queue	HIGH
Callback Queue	LOW

Promises use microtask queue.

Promise Chaining

Instead of nested callbacks:

```
login()
  .then(getProfile)
  .then(getPosts)
  .then(getComments)
  .catch(handleError);
```

Cleaner and readable.

How Chaining Works

Every `.then()` returns:

A NEW Promise

Example

```
Promise.resolve(10)

  .then((num) => {
    return num * 2;
  })

  .then((num) => {
    return num + 5;
  })

  .then((result) => {
    console.log(result);
  });
```

Output

```
25
```

Internal Flow

```
10
↓
20
↓
25
```

Each `.then()` passes result to next `.then()`.

Returning Promise from then()

```
function double(num) {

  return new Promise((resolve) => {

    resolve(num * 2);

  });

}

double(5)

  .then((data) => {
    return double(data);
  })

  .then((result) => {
    console.log(result);
  });
```

Output

```
20
```

Promise Flattening

JavaScript automatically unwraps returned promises.

This is called:

Promise Resolution Procedure

Error Handling in Promises

Example

```
Promise.resolve()

  .then(() => {
    throw new Error("Something wrong");
  })

  .catch((error) => {
    console.log(error.message);
  });
```

Output

```
Something wrong
```

Important Rule

Errors automatically travel down chain.

Example

```
Promise.resolve()

  .then(() => {
    return 10;
  })

  .then(() => {
    throw new Error("Failed");
  })

  .then(() => {
    console.log("Never runs");
  })

  .catch((err) => {
    console.log(err.message);
  });
```

Output

```
Failed
```

---

finally()

Runs always.

```
Promise.resolve("Success")

  .finally(() => {
    console.log("Cleanup");
  });
```

Important

finally does NOT receive value.

Promise Static Methods

---

1. Promise.resolve()

Creates resolved promise.

```
Promise.resolve(100)
  .then(console.log);
```

Output:

```
100
```

---

2. Promise.reject()

Creates rejected promise.

```
Promise.reject("Error")
  .catch(console.log);
```

---

3. Promise.all()

Runs multiple promises together.

If ANY fails:

Entire promise fails.

Example

```
const p1 = Promise.resolve(1);
const p2 = Promise.resolve(2);
const p3 = Promise.resolve(3);

Promise.all([p1, p2, p3])

  .then((data) => {
    console.log(data);
  });
```

Output

```
[1, 2, 3]
```

Internal Working

Promise.all waits for ALL promises.

Failure Example

```
const p1 = Promise.resolve(1);
```

```
const p2 = Promise.reject("Failed");

Promise.all([p1, p2])

  .catch(console.log);
```

Output:

```
Failed
```

---

4. Promise.race()

Returns first completed promise.

Example

```
const p1 = new Promise((resolve) => {
  setTimeout(() => resolve("Fast"), 1000);
});

const p2 = new Promise((resolve) => {
  setTimeout(() => resolve("Slow"), 3000);
});

Promise.race([p1, p2])

  .then(console.log);
```

Output:

```
Fast
```

---

5. Promise.allSettled()

Returns ALL results.

Even failed ones.

Example

```
Promise.allSettled([

  Promise.resolve("Success"),

  Promise.reject("Failed")

])

.then(console.log);
```

Output:

```
[
  { status: "fulfilled", value: "Success" },
  { status: "rejected", reason: "Failed" }
]
```

---

6. Promise.any()

Returns first SUCCESSFUL promise.

Ignores failures.

Example

```
Promise.any([

  Promise.reject("Error"),

  Promise.resolve("Success")

])

.then(console.log);
```

Output:

```
Success
```

Promise vs setTimeout

Very important interview topic.

Example

```
setTimeout(() => {
  console.log("Timeout");
}, 0);

Promise.resolve().then(() => {
  console.log("Promise");
});
```

Output:

```
Promise
Timeout
```

Why?

Promises → Microtask Queue
setTimeout → Callback Queue

Microtask has higher priority.

Async Await

Built on promises.

Makes async code look synchronous.

Without async/await

```
fetchData()

  .then((data) => {
    console.log(data);
  })

  .catch(console.log);
```

With async/await

```
async function run() {

  try {

    const data = await fetchData();

    console.log(data);

  } catch (error) {

    console.log(error);

  }

}
```

---

async Function

Always returns promise.

Example

```
async function test() {
  return 5;
}
```

Actually returns:

```
Promise.resolve(5)
```

---

await Keyword

Pauses async function until promise resolves.

Example

```
const result = await promise;
```

Important

`await` only pauses:

`that async function`

NOT entire JavaScript.

Creating Delays with Promise

```
function delay(ms) {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}
async function run() {
  console.log("Start");
  await delay(2000);
  console.log("After 2 seconds");
}
run();
```

Promise States Internally

Internally promise stores:

```
[[PromiseState]]
[[PromiseResult]]
```

Example

```
Promise { <pending> }
Promise { <fulfilled>: 10 }
Promise { <rejected>: Error }
```

Important Interview Questions

Difference Between Callback and Promise

Callback	Promise
nested	chaining

Callback	Promise
callback hell	cleaner
hard error handling	catch
inversion of control	better control

Difference Between Promise and Async Await

Promise	Async Await
then/catch	try/catch
chaining	synchronous style
harder to read	cleaner

Difference Between resolve and reject

resolve	reject
success	failure
then()	catch()

Can Promise Resolve Twice?

NO.

Only first resolve/reject works.

Example

```
new Promise((resolve, reject) => {

  resolve(1);

  resolve(2);

  reject(3);

})
.then(console.log);
```

Output:

```
1
```

Promise Memory Trick

Think:

```
Promise = Future Result Box
```

- pending → waiting

- fulfilled → success
- rejected → failure

Final Ultimate Summary

Concept	Meaning
Promise	future value
Pending	waiting
Fulfilled	success
Rejected	failure
resolve	success function
reject	error function
then	handles success
catch	handles errors
finally	always runs
Promise.all	wait all
Promise.race	first completed
Promise.any	first success
Promise.allSettled	all results
async/await	cleaner promises
Microtask Queue	promise queue